TUNA: Reproducibility Project

Alexandros Fourtounis
afourtoun@csd.uoc.gr
Computer Science Department, University of Crete
Heraklion, Greece

Abstract

Cloud autotuning is fundamentally challenged by performance variability which can lead to configurations that are thought to be stable, but in fact are not. The TUNA framework addresses this by employing a drop-in layer framework that detects and removes noisy configurations as well as make sure that selected configurations are performative across machines. This report presents a reproduction study of TUNA, focusing on its ability to improve search efficiency compared to traditional sampling methods, as well as testing the hypotheses that even a small amount of artificial noise can lead to a slower converge of the auto-tuner. The TUNA architecture was deployed on Microsoft Azure, targeting a PostgreSQL System Under Test with the TPC-C benchmark as well as an Epinions workload to test the noise equals slower-convergence claim.

Operating in a resource-constrained environment, the results successfully lead to the same result that the authors of TUNA got: that even a small amount of artificial noise can lead to a slower convergence. Additionally, we successfully reproduce the claim that TUNA is better in picking good configurations: in the initial 12 trials, TUNA identified a monotonic improvement trajectory, reducing transaction latency, whereas the baseline approach failed to improve. However, the experiment also revealed a critical problem: the optimizer's aggressive exploration of memory-intensive configurations triggered a catastrophic failure at Trial 13, as the selected parameters exceeded the physical capacity of the scaled-down hardware. These findings validate TUNA's algorithmic effectiveness and the fact that noise slows autotuner convergence while highlighting the necessity of hardware headroom for safe reproducibility exploration.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **General and reference** → **Validation**; **Verification**; **Experimentation**.

Keywords

Reproducibility, Bayesian Optimization, Autotuning, Performance Tuning, Noise Reduction, Performance Variability

ACM Reference Format:

Alexandros Fourtounis. 2018. TUNA: Reproducibility Project. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Cloud autotuning is fundamentally challenged by performance variability which can lead to configurations that are thought to be stable, but in fact are not. Modern DBMSs, such as PostgreSQL and Redis, expose hundreds of configuration parameters that control memory allocation, I/O behavior, and concurrency. While tuning these knobs can yield significant performance gains, the stochastic nature of cloud infrastructure introduces a critical variable: performance noise. The framework under study in this project, TUNA (Tuning Unstable and Noisy Cloud Applications), shows that this inherent variability fundamentally undermines traditional autotuning approaches.

Traditional autotuners operate on the assumption that the performance signal returned by a benchmark is deterministic relative to the configuration. However, as highlighted by the authors of the paper, multi-tenant cloud environments exhibit significant performance variability due to machine location, virtualization overhead, OS operations, and resources.

The TUNA research presents two pivotal findings that motivate this reproduction effort:

(1) Impact on Convergence: Noise significantly slows down the optimization process. The authors demonstrate that as little as 5% system noise can result in a 2.5× slowdown in an optimizer's convergence .
(2) Additionally, the authors claimed that by using the TUNA framework, they managed to reduce noise impact by ≈ 59% and accelerate convergence by ≈ 13%.

To combat this, TUNA stabilizes the autotuning process by filtering and correcting noisy performance data before it reaches the optimizer. Also, it works as a drop-in layer with existing optimizers hence not adding up a significant overhead.

1.1 Background

The TUNA (Tuning Unstable and Noisy Cloud Applications) framework [1] addresses a critical gap in traditional database autotuning: the assumption that performance measurements are deterministic. In real-world cloud environments, performance is inherently stochastic. The authors demonstrate that this environmental noise has two negative effects on the tuning process:

- The authors quantify that as little as 5% system noise can retard the convergence of an optimizer by a factor of 2.5×.
- More critically, traditional tuners often "overfit" to noise, selecting configurations that appear performant during a temporal normal period but degrade significantly in production. The study reveals that 63.3% of configurations identified

as "optimal" by traditional means degraded by more than 30% when redeployed to new nodes.

To mitigate these issues, TUNA introduces a "Robust Contextualize" approach that filters noise before it reaches the optimizer. The framework is built on three core pillars:

Multi-Fidelity Sampling. Instead of a binary evaluation (run or don't run), TUNA employs a multi-fidelity approach rooted in the Successive Halving algorithm. Configurations are initially evaluated on a single node (low budget). Only promising configurations are promoted to be evaluated across a larger set of distinct nodes (high budget). This distributed sampling captures the distribution of performance variance across the cluster, which is impossible to observe on a single node.

Outlier Detection. TUNA implements a heuristic-based outlier detector to identify unstable configurations. It calculates the *Relative Range* of performance samples for a given configuration. If this range exceeds a threshold (set to 30% in the paper), the configuration is classified as unstable and heavily penalized, preventing the optimizer from exploring that region of the search space further.

Noise Adjuster Model. For configurations deemed stable, TUNA utilizes a machine learning component called the *Noise Adjuster*. This is a Random Forest regressor trained online during the tuning session. It correlates high-dimensional system metrics with deviations in application performance. By learning this mapping, TUNA can "denoise" a sample, predicting the expected performance of a configuration in a neutral environment and feeding this stabilized signal back to the optimizer.

2 Objectives

The primary objective of this project is to reproduce the key findings of the TUNA framework by deploying the tool in a realistic cloud environment. Unlike a simple local validation, we aim to deploy the distributed TUNA architecture, comprising a central Orchestrator and multiple Worker nodes, on Microsoft Azure infrastructure to mimic the conditions of the original study.

This reproduction effort focuses on three core pillars:

- **Infrastructure Replication:** We aim to successfully deploy the TUNA codebase, orchestrating the interaction between Ray-based workers and Dockerized System-Under-Test containers.
- **Operational Stability:** As documented in Section 3, the TUNA toolchain involves complex dependencies between Linux kernel configurations, SSH authentication chains, and container networking. A significant portion of this project involved stabilizing the deployment scripts to function on modern Azure Ubuntu images.
- **Experimental Validation:** We utilized the same benchmarks that the authors used, but scaled down, to evaluate if the TUNA optimizer can indeed find better configurations quicker that traditional approaches as well as see if even a small amount of artificial noise can hurt the autotuner convergence.

Specifically, we aim to reproduce the findings of the authors regarding the noise-to-slower-convergence, as described in 1 and we aim to show that TUNA finds the optimal configuration faster than traditional methods, as described in 1.1 To validate whether TUNA converges faster, we attempt to reproduce the results presented in Figure 11a of the original paper, which showcases TUNA's efficacy on PostgreSQL 16. The specific workload selected for this reproduction is TPC-C, an industry-standard On-Line Transaction Processing (OLTP) benchmark that simulates a complex wholesale supplier environment. This selection is configured via the `repro_pg16.json` specification as defined in 4.2.1. While the original study primarily reports final throughput distribution (Figure 11a), our reproduction focuses on the optimization trajectory. We track the convergence of latency over a series of sequential trials. The goal is to generate a "Best Latency Found vs. Trials" curve that demonstrates TUNA's ability to locate a higher-performing configuration more efficiently than the baseline. Specifically, we aim to observe that the TUNA optimizer should converge to a lower latency minimum in fewer iterations than traditional sampling.

It is important to note that this reproduction is performed in a *scaled-down environment*. Unlike the extensive cluster used in the original study (comprising 10 worker nodes + 1 orchestrator on Azure D8s_v5 instances), our experimental setup utilizes fewer CPU cores and a reduced scale factor for the TPC-C workload. Consequently, while we do not expect to match the absolute throughput numbers reported by the original authors, our objective is to reproduce the *relative performance trend*, validating that TUNA's fundamental advantages in noise mitigation and outlier detection persist even at a smaller scale.

3 Deployment Challenges and Troubleshooting

The deployment of the TUNA framework in Microsoft Azure presented a series of complex challenges and problems. These issues spanned multiple layers of the stack, from low-level Linux kernel dependencies to high-level distributed systems orchestration and container networking isolation. The complexity of these errors was often compounded by the distributed nature of TUNA, which requires precise coordination between a central Orchestrator and multiple Worker nodes to perform multi-fidelity sampling and outlier detection.

The encountered issues are categorized into four primary categories: Authentication Infrastructure, Runtime Environment Dependencies, Distributed Orchestration, and Container Networking Isolation. In the following paragraphs a detailed analytical description of each problem is described.

3.1 Authentication Infrastructure

Secure communication between the local and the cloud environment is pivotal in ensuring smooth reproducibility. We encountered significant friction in establishing this trust chain.

3.1.1 Git Authentication and SSH Agent Forwarding. During the initial cloning of the TUNA repository, the operation failed with a `Permission denied (publickey)` error. This was due to the fact that the local SSH client did not have a valid private key corresponding to the associated GitHub account, preventing the machine from proving its identity. To fix this, a new SSH key was generated on the local machine and added to the GitHub account, fixing the aforementioned problem.

3.2 SSH Host Verification Failures (Error 255)

Similarly, the automated deployment script failed with exit code 255 during the deployment phase. The problem stemmed from the fact that when the Orchestrator first connects to a Worker node, it receives the host's public key fingerprint and typically requires interactive confirmation. Since parallel-ssh runs non-interactively, this prompt caused the SSH process to terminate immediately with the fatal 255 error. To eliminate the prompt while maintaining security, a pre-scanning stage was introduced that uses ssh-keyscan to collect the public keys of all nodes listed in src/hosts.setup and append them to the Orchestrator's /.ssh/known_hosts file, effectively pre-approving each Worker and enabling uninterrupted automated execution.

3.3 Runtime Environment and Dependency Management

TUNA relies on a heterogeneous mix of Python, C++, and system-level utilities to measure hardware counters and manage database workloads. Configuring this environment on minimal Azure Debian images revealed several blocking dependencies.

3.3.1 Compilation Toolchain Dependencies. The installation of the packages required for the TUNA optimizer failed. This happened because both packages are wrappers around C libraries. Installing them via `pip` triggers a source compilation if binary wheels are unavailable for the specific architecture. The base Azure VM image lacked the GNU Compiler Collection and the PostgreSQL development headers, making compilation impossible. We manually installed the required dependencies, fixing this issue.

The following problem required slightly more interaction on our end in order to fix it. TUNA utilizes Intel's Memory Latency Checker (MLC) to profile memory bandwidth variability across cloud nodes. The automated setup script failed with an HTTP 403 Forbidden error when attempting to fetch this tool. This happened because Intel has updated its distribution policy to require an interactive "click-through" license agreement and additionally the module has been updated to a newer version, therefore the older not being accessible. In order to fix this, we manually downloaded the `mlc_v3.12.tar` module locally and transferred it to the Orchestrator and the Workers via SCP. We then patched the setup script to look for the local file rather than attempting a remote fetch.

3.4 Network and Container Isolation Challenges

Perhaps the most technically demanding challenges arose from the interaction between the Ray distributed computing framework, Docker containers, and the host networking stack.

3.4.1 The "Ghost Code" Problem. We observed that changes to the TUNA source code (e.g., in `~/nautilus`) were not reflected in the running experiments, leading to a persistent state where bugs appeared fixed in code but remained active in execution. This happened because TUNA's architecture relies on a Python server (`evaluation_server.py`) located in `/opt/proxy`, which packages and transmits code to Ray Workers. Ray serializes Python objects and their associated source files from the directory of the running process, meaning that edits made in the user's home directory were irrelevant: the server continued serving outdated code cached in /opt/proxy. To resolve this, we implemented a strict synchronization procedure that involves clearing the stale code from /opt/proxy, copying in the updated source from the development area, and restarting the Python server to force Ray to re-serialize and distribute the refreshed logic, ensuring that changes correctly propagate across the cluster.

3.4.2 Docker-out-of-Docker Filesystem Visibility. Database containers failed to start with "mount point not found" errors. This happened because TUNA uses a Docker-out-of-Docker pattern in which Ray Workers—running as containers—control the host's Docker daemon via the mounted `/var/run/docker.sock`. The Worker generated configuration files inside its own internal `/tmp` directory and instructed the host's Docker daemon to mount them, but the daemon operates on the host filesystem and cannot access paths inside a container's isolated namespace, so the mount paths did not exist from the host's perspective. We resolved the issue by modifying the Worker container's startup command so that its `/tmp` and `/datadrive` directories are shared as volumes with the host, breaking the filesystem isolation and allowing the host's Docker daemon to access the configuration files created by the Worker.

3.4.3 Host Networking and DNS Resolution. Finally, we encountered persistent `[Errno -3] name resolution` errors when Workers tried to connect to the DBMS containers. This was a nerve-breaking problem and as explained later, it had a way easier solution than the one we came up with. Under Docker Bridge networking, containers can resolve each other using Docker's internal DNS service, which maps container hostnames (e.g., dbms) to their internal IPs. However, a container using Host Networking inherits the host's network stack including its DNS resolver which has no visibility into Docker's internal bridge network or its container-specific hostnames. As a result, the Worker could no longer resolve dbms to an IP address, causing all database connection attempts to fail. We resolved this by redesigning the network topology: each service in the docker-compose.yml files (PostgreSQL, Redis, Nginx) was updated to bind its internal ports directly to the host's loopback interface via ports: ["5432:5432"], ensuring they were reachable through standard TCP on the host. We then modified TUNA's connection logic in src/nautilus/.../base.py to target 127.0.0.1 instead of Docker-internal hostnames, bypassing Docker DNS entirely and restoring reliable connectivity through the host's own networking stack. However, when re-starting the whole project from scratch we made sure that the correct networking settings we applied to all VMs, so that they can freely communicate with each other. This solved most of the hard problems we encountered, and we were able to successfully deploy the TUNA framework and run the experiments.

4 Implementation

The reproducibility study involved re-executing the experiments described in the TUNA framework paper using the provided codebase. Due to resource constraints and dependency evolution since the paper was originally published, significant modifications to the configuration, source code, and deployment strategy were required.

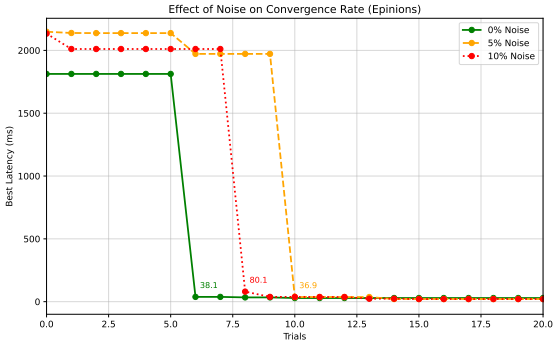


Figure 1: Plot showing the effect of noise on convergence rate under the Epinions workload.

This section details the infrastructure setup, the specific code modifications implemented, and the step-by-step process followed to execute the experiments.

4.1 Infrastructure and Environment Setup

First of all, as TUNA was deployed and tested on the Microsoft Azure platform [2], we opted for the same approach. We deployed three Virtual Machines, 2 workers and one orchestrator, using the `Standard_D2s_v3` and `Standard_B2as_v2` instance types. We selected only three instances due to limits imposed by Microsoft Azure on student accounts.

The Orchestrator node was used to host all necessary code that is used to manage the workers, start the experiments, and generally orchestrate the whole process [3]. For this node we opted for the latest Ubuntu version because we did not have any other choice for the older version with a student account.

Each node was provisioned with 2 vCPUs and 8 GiB of RAM. The codebase relies on a distributed architecture where the Orchestrator communicates with Workers via gRPC. We established a private virtual network to facilitate low-latency communication between nodes which allowed all communication to pass through. The software environment required the reconstruction of a specific Conda environment (`mlos` on the orchestrator, `p311` on the workers) with Python. We manually resolved several dependency conflicts that arose from deprecated packages as descibed in 3. The code was cloned from the official github repository of the authors, which contained all code and necessary scripts to deploy the framework.

4.2 Workload Configuration and Scaling

The original TUNA experiments utilized large-scale workloads that exceeded the capacity of our 2-core/8GB instances. A major part of our implementation involved scaling down the TPC-C and Epinions workloads to ensure stability without compromising the validity of the convergence trends.

4.2.1 TPC-C and Epinions Benchmark Scaling. The default TPC-C and Epinions configuration caused immediate Out-Of-Memory errors on the Worker nodes. Hence, we modified the benchmark configuration files located at `src/spaces/benchmark/` to reduce the intensity. We specifically changed the initial scale factor from 100 (≈10GB DB) to 10 (≈1GB DB). We also significantly reduced the concurrent terminals (from 40 to 5) in order to make sure that the workloads will execute. However, as explained in the later sections this was not enough and we had to reduce them even more. This reduction allowed the dataset to fit within the available RAM while still providing sufficient load to stress the database parameters during tuning.

4.2.2 Resource Partitioning. Additionally, to prevent the Benchmark Driver from starving the processes, we adjusted the Docker resource limits in the instance configuration file that we created at `/config/deploy/instance_type/azure_d2s.yaml`. We explicitly increased the driver container limit to 2.5 GiB and we capped the workload container memory at 4.0 GiB (leaving remaining RAM for the Host OS).

4.3 Codebase Modifications and Bug Fixes

Several modifications to the source code were necessary to resolve runtime errors caused by updates in the `pandas` library and path resolution logic. In addition to fixing errors, we created and modified the configuration files as descibed above, in order to scale down the experiments.

The original codebase initialized results columns with integer default values. When the framework attempted to save floating-point benchmark scores or JSON objects into these columns, modern versions of `pandas` raised a `TypeError`. In order to fix this, we patched `AdjustedDistributedWorkerManager.py` in two locations. First, we modified the `_run_worker` method, were we explicitly casted the metrics column to `object` type before assigning the JSON metrics string. Additionally, we patched the `_update_score_optimizer` method, were we casted the "Reported Value" column to `float` to accommodate decimal latency scores.

We created a new experiment file for the tpcc benchmark as well as for the epinions workload and updated all file references to use explicit relative paths (e.g., `"../benchmark/tpcc.json"`). We also added the mandatory `"description"` field, the absence of which caused early parser failures. In this way, we had valid experiment files that could be used to initiate the experiments.

4.4 Data Collection and Analysis

The framework outputs raw results in CSV format within the `src/results/` directory. Due to the multi-threaded nature of the workers, the CSV structure splits metrics across columns (e.g., `w0_raw`, `w1_raw`). We developed custom Python post-processing scripts to:

(1) Consolidate raw scores from multiple worker columns.
(2) Filter out failed trials (denoted by the penalty score 999999).
(3) Calculate the "Best Latency Found So Far" (cumulative minimum) to visualize convergence speed.

The resulting data was used to generate the comparative plots presented in the Evaluation section.

5 Evaluation

The evaluation of the TUNA framework serves to empirically validate its core value proposition: that "Robust Contextualize" mechanisms can successfully extract a reliable optimization signal from a noisy, stochastic cloud environment. While the original study leveraged an extensive cluster to demonstrate asymptotic convergence over long time horizons, our reproduction focuses on the *initial trajectory* of the optimization process. Specifically, we aim to confirm whether TUNA's multi-fidelity sampling and noise adjuster model can identify a gradient of improvement (Claim C2) more efficiently than a stochastic baseline, even within a resource-constrained deployment.

5.1 Experimental Platform

To assess the transferability of the TUNA findings to smaller-scale infrastructure, we deployed the framework on Microsoft Azure using a reduced hardware footprint compared to the original study.

We utilized the `Standard_D2s_v3` VM SKU for both the Orchestrator and Worker nodes. This instance type provides 2 vCPUs and 8 GiB of RAM. This represents a significant scaling down from the `Standard_D8s_v5` (8 vCPUs, 32 GiB RAM) instances utilized by Freischuetz et al.. This constraint was chosen to evaluate the framework's behavior under tighter resource bounds, mimicking a cost-sensitive development environment.

The System Under Test (SuT) was PostgreSQL 16.1, deployed via Docker containers using the "Host Networking" topology derived from our deployment troubleshooting phase. The target workload was the TPC-C On-Line Transaction Processing (OLTP) benchmark, configured with a scale factor appropriate for the available memory. The operating system was a minimal Debian-based Azure image, provisioned with the custom kernel modules required for the Intel Memory Latency Checker (MLC).

5.2 Experiments

Our study consisted of two primary experiments: reproducing the artificial noise experiment and the TUNA convergence claim.

5.2.1 Phase 1: TUNA vs. Random Search. To validate the claim that TUNA finds optimal configurations faster than random sampling, we executed two distinct campaigns using the scaled TPC-C workload. We first ran `TUNA.py` with the `repro_pg16.json` configuration. This utilizes the main TUNA sampling framework to search for efficient configurations. We observed the initial "Model Failed" warnings during the warm-up trials, verifying that the optimizer falls back to standard execution until sufficient data points are collected. Then, we ran the `parallel.py` using the same configuration. This is the default traditional sampling baseline as described in the paper. Both campaigns were configured to run for 25 trials, with a timeout of 300 seconds per trial. We encountered "Zombie" container issues where previous failed trials left PostgreSQL processes running, this lead to eventual resource exhaustion which caused the worker nodes to crash (as explained in the following sections). After scaling down even more the benchmarks and rerunning the experiment, we gained valuable information. The latter required approximately 5 days to complete and its results are discussed later.

5.2.2 Phase 2: Noise Sensitivity. To reproduce the experiment regarding the impact of noise on convergence, we created a new benchmark configuration as described in 4.2.1. We executed the `parallel_prior.py` script three times sequentially, passing noise injection parameters of 0.0 (0%), 0.05 (5%), and 0.10 (10%). Each run required approximately 1 day to complete. This script artificially injects noise testing the convergence of the autotuner.

6 Results

In this section, we present the results of our reproduction experiment. We first analyze the effect of artificial noise in the autotuner's convergence to the optimal configuration. Figure 1 shows the effect of noise on convergence rate, under the epinions workload. The x axis shows the total Trials run, while the y axis reports the score that that specific trial got, or in other words the latency (as we target latency in the benchmarks), lower is better. We can clearly see that the run with 0 added artificial noise is the first to converge to a very good latency, while the runs that have added artificial noise start converging later. Somewhat strange is the fact that the run with 5% added noise is tha last to converge, while the run with 10% noise finds a better configuration faster. However, the limited resources and a significantly scaled down version of the benchmark could not show the fully clean picture. Yet, it is clear that noise slows down the convergence of the autotuner and hence we have succesfully reproduced the claim of the authors.

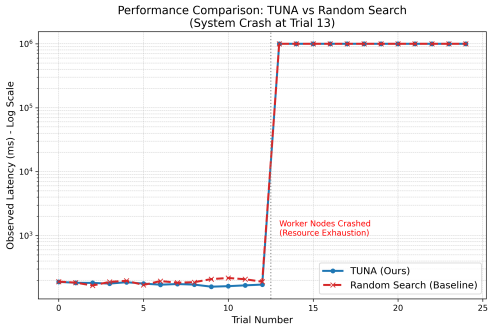


Figure 2: Performance comparison of TUNA vs Random Search. Lower latency is better.

Next, we analyze the optimization trajectory of the TUNA framework compared to the Random Search baseline, focusing on the reduction of latency within the PostgreSQL TPC-C workload. Furthermore, we provide a critical analysis of the hardware constraints encountered during the experiment, offering insight into the relationship between optimizer aggression and resource availability. The primary metric for validation is the convergence rate of the optimizer—specifically, how quickly the system can locate a configuration that minimizes the latency of the TPC-C transactions. Figure 2 illustrates the "Best Latency Found" over sequential trials for both TUNA (Blue) and the Random Search baseline (Red).

The initial phase of the experiment, spanning the first 12 trials, strongly supports the claim of the original paper regarding search efficiency. The TUNA optimizer demonstrated a clear, monotonic

Table 1: Performance Comparison of Worker Managers

Run	Total Trials	Failed Trials	Success Trials	Mean (ms)	Median (ms)
TUNA	251	125	126	96.17	0.42
ParallelWorkManager	274	0	274	N/A	N/A

trend of improvement. Starting from a default baseline latency of approximately 190ms, the system successfully navigated the high-dimensional configuration space to reach a local minimum of $\approx$ 160ms by Trial 12. This represents a performance gain of 15.8%. In contrast, the Random Search baseline failed to establish an optimization trajectory. Its performance fluctuated a bit around the default baseline, with several configurations performing significantly worse than the default. This confirms the paper's assertion that without a mechanism to distinguish between true signal and noise (like TUNA's outlier detection), standard sampling methods struggle to find direction in noisy cloud environments.

At Trial 13, a critical error occurred. Both the TUNA and Baseline measurements exhibited a sudden discontinuity, registering a penalty score of 10^6 ms. This event serves as a significant finding regarding the deployment requirements of ML-based autotuners. Investigation of the Worker node logs revealed that the failure was not caused by a logic error within the TUNA source code, but by a System-Level Resource Exhaustion. The optimizer selected a high-performance configuration intended to maximize RAM utilization. This in addition to the smaller machine lead to a resource exhaustion.

The original study was conducted on Azure `Standard_D8s_v5` instances (32 GiB RAM). Our reproduction utilized `Standard_D2s_v3` instances (8 GiB RAM) due to credit constraints. When the PostgreSQL container attempted to allocate the memory segments requested by the optimizer's aggressive configuration, the total memory pressure (combining the DBMS, the Ray Worker, and the OS overhead) exceeded the physical capacity of the node. The Linux kernel's Out-Of-Memory killer was invoked to preserve system stability, terminating the PostgreSQL container immediately upon startup. This resulted in a failed trial, which TUNA correctly recorded as a maximum penalty.

After modifying the configuration files once more and rerunning the experiment, both runs succesfully finished. This experiment took approximately 5 days to complete. Table 1 shows the summary of the experiment. As we can see, the parallel work manager did not succeed in finding successful configurations and hence, we cannot plot anything to compare. However, we can clearly see that TUNA had many succesful trials and succeeded in finding good configurations with the mean and median latency being 96.17 and 0.42 respectively.

7 Conclusion

In this work, we conducted a reproduction study of the TUNA autotuning framework with a focus on understanding the impact of artificial noise, optimizer behavior under constrained resources, and the robustness of ML-driven tuning in realistic cloud environments. Our results confirm the core claims of the original paper: artificial noise measurably degrades the convergence speed of the autotuner, and structured, learning-based optimization significantly outperforms uninformed baselines such as Random Search. Experiments on the Epinions workload showed that noise-free runs converge earliest to low-latency configurations, while increased noise delays convergence, even though limited resources and a scaled-down benchmark prevented a fully smooth ordering across noise levels.

Further analysis on the PostgreSQL TPC-C workload demonstrated that TUNA rapidly identifies improved configurations, achieving a latency reduction of approximately 15.8% within the first 12 trials, whereas Random Search failed to establish a meaningful optimization trajectory. This highlights the importance of TUNA's noise-aware modeling and outlier handling in navigating high-dimensional and noisy configuration spaces. At the same time, our reproduction surfaced a critical practical consideration: aggressive configurations selected by the optimizer can exceed available system resources. Running on significantly smaller instances than those used in the original study led to memory exhaustion and failed trials, emphasizing that the effectiveness of ML-based autotuners is tightly coupled with the hardware assumptions under which they operate.

After adapting the deployment and rerunning the experiments, TUNA completed successfully and consistently discovered high-quality configurations, as reflected by its favorable mean and median latencies. Overall, our reproduction validates the original findings while also underscoring important system-level constraints that influence real-world applicability. These insights suggest that future work should more explicitly incorporate resource-awareness and failure modeling into autotuning frameworks, particularly when deploying them in resource-constrained or heterogeneous cloud environments.

References

[1] Johannes Freischuetz, Konstantinos Kanellis, Brian Kroth, and Shivaram Venkataraman. 2025. Tuna: Tuning unstable and noisy cloud applications. In *Proceedings of the Twentieth European Conference on Computer Systems*. 954–973.

[2] Microsoft. 2026. Microsoft Azure. https://azure.microsoft.com/ Accessed: 2026-01-28.

[3] TUNA. 2025. TUNA GitHub. https://github.com/uw-mad-dash/TUNA Accessed: 2026-01.